Waterpark & Amusement Park Management Software

Complete Developer Reference Document

Version 1.0 | Stack: Next.js 14 + PostgreSQL 16 + Redis 7 + Docker | Server: Ubuntu 24.04 LTS ARM64

1. Project Overview

1.1 What We Are Building

A full-stack, multi-role **Waterpark & Amusement Park Management Software** — a production-grade SaaS-ready platform to manage every operational and revenue function of a waterpark or amusement park.

The system serves **three user roles**:

- **Admin** — full access: bookings, reports, staff, CRM, settings
- **Employee (Staff)** — POS terminal, ride operations, locker management, attendance
- **Guest (Customer)** — online booking, payment, my-account, loyalty points

1.2 Business Goals

Goal	Implementation
Increase pre-booking revenue	Online ticket booking with payment gateway + manual UPI
Reduce entry queue time	QR code scan check-in at gate
Increase per-guest spend	RFID wristband cashless F&B + activity booking
Capture leads	CRM with lead pipeline, WhatsApp/SMS follow-up
Automate communication	Automated booking confirmation, reminders, birthday offers
Real-time park management	Live capacity, ride status, UPI queue on admin dashboard
Staff accountability	Shift scheduling, attendance, role-based access
Data-driven decisions	Revenue reports, footfall analytics, ride popularity

2. System Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    INTERNET / USERS                      │
└──────────────────────────────────────────────────────────┘
                        │
                        │ HTTPS (443)
┌──────────────────────▼───────────────────────────────────┐
│              NGINX (Host — Ubuntu 24.04)                 │
│   SSL Termination | Reverse Proxy | Static Cache         │
│   WebSocket Upgrade | Security Headers                   │
└──────────────────────────────────────────────────────────┘
                        │
                        │ HTTP (127.0.0.1:3000)
┌──────────────────────▼───────────────────────────────────┐
│          NEXT.JS 14 APP (Docker Container)               │
│   App Router | API Routes | Server Components            │
│   NextAuth JWT | Middleware RBAC | Prisma ORM            │
└──────────────────────────────────────────────────────────┘
          │                          │
          │                          │
┌─────────▼────────┐        ┌────────▼──────────┐
│  POSTGRESQL 16   │        │     REDIS 7       │
│  (Docker)        │        │  (Docker)         │
│  Primary DB      │        │  Sessions + Cache  │
│  127.0.0.1:5432  │        │  127.0.0.1:6379   │
└──────────────────┘        └───────────────────┘

External Services (API calls from Next.js):
  Razorpay         → Payment gateway
  MSG91            → SMS OTP + bulk SMS
  Gupshup          → WhatsApp messaging
  SendGrid         → Transactional email
  Firebase FCM     → Push notifications
```

2.2 Infrastructure

Component	Technology	Version	Location
Server OS	Ubuntu LTS	24.04 ARM64	OCI Ampere A1 / VPS
Reverse Proxy	Nginx	Latest stable	Host (apt)
SSL	Let's Encrypt (Certbot)	Auto-renew	Host
App Runtime	Node.js	20 LTS Alpine	Docker container
App Framework	Next.js	14.2.x	Docker container
Database	PostgreSQL	16 Alpine	Docker container
Cache/Session	Redis	7 Alpine	Docker container
Container Mgmt	Docker Compose	v2 plugin	Host
CI/CD	GitHub Actions	—	GitHub → SSH deploy

Component	Technology	Version	Location
Monitoring	Docker stats + htop	—	Host
Backup	pg_dump cron	Daily 2 AM	Host cron job

2.3 Port Map

Service	Port	Bound To	Accessible From
Nginx HTTP	80	0.0.0.0	Public internet
Nginx HTTPS	443	0.0.0.0	Public internet
Next.js app	3000	127.0.0.1	Nginx only
PostgreSQL	5432	127.0.0.1	App container only
Redis	6379	127.0.0.1	App container only

2.4 Request Flow

```
Guest visits https://yourdomain.com
  → Nginx receives on :443
  → SSL termination with Let's Encrypt cert
  → proxy_pass to 127.0.0.1:3000
  → Next.js App Router handles request
  → Server Component fetches from PostgreSQL via Prisma
  → OR API Route handles POST/PUT/DELETE
  → Redis checked first for cached responses
  → Response returned to Nginx → to client
```

3. Technology Stack

3.1 Frontend

Technology	Purpose	Why Chosen
Next.js 14 App Router	Full-stack React framework	SSR, API routes, file-based routing
TypeScript	Type safety	Catch errors at build time
Tailwind CSS 3	Utility-first styling	Fast UI, no CSS files to manage
Lucide React	Icon library	Lightweight, consistent icons
Recharts	Charts/analytics	React-native, responsive charts
React Hot Toast	Notifications	Simple toast system
React QR Code	QR display	Booking confirmation QR
TanStack Query v5	Data fetching + cache	Optimistic updates, background refresh
date-fns	Date manipulation	Lightweight, tree-shakeable

3.2 Backend

Technology	Purpose
Next.js API Routes	REST API endpoints
Prisma ORM 5	Database access, migrations, type-safe queries
NextAuth v4	Authentication (JWT strategy)
Zod	Input validation on all API routes
bcryptjs	Password hashing (12 rounds)
IORedis	Redis client for sessions and cache
Razorpay SDK	Payment order creation and verification
@sendgrid/mail	Transactional email
firebase-admin	Push notification via FCM
Axios	HTTP client for external API calls

3.3 Database

Technology	Purpose
PostgreSQL 16	Primary relational database
uuid-ossp extension	Generate UUID primary keys
pg_trgm extension	Full-text search on guest/lead names
Prisma Migrate	Schema migrations with version history
pg_dump	Daily backup compressed with gzip

4. Role & Permission Matrix

4.1 User Roles

Role	Description	Login Method
ADMIN	Park owner / manager — full system access	Mobile + Password
EMPLOYEE	Park staff — limited to their assigned module	Mobile + Password
USER	Guest / customer — booking and account access	Mobile + OTP

4.2 Employee Sub-Roles

Sub-Role	Module Access
TICKET_COUNTER	POS terminal, booking lookup, check-in
FB_STAFF	F&B outlet orders, kitchen display
RIDE_OPERATOR	Ride status update, access log
MAINTENANCE_TECH	Work orders, safety inspections
LOCKER_ATTENDANT	Locker assign/release
SALES_EXECUTIVE	CRM leads, campaign viewer
SECURITY_STAFF	Entry check-in, incident reports
EVENT_COORDINATOR	Event packages, corporate bookings

4.3 Route Protection

Route Pattern	Required Role	Middleware Check
/admin/*	ADMIN only	middleware.ts
/staff/*	ADMIN or EMPLOYEE	middleware.ts
/guest/my-account/*	Any authenticated USER	middleware.ts
/api/v1/admin/*	ADMIN only	API route guard
/api/v1/staff/*	ADMIN or EMPLOYEE	API route guard
/api/v1/guest/*	Authenticated USER	API route guard
/api/v1/webhooks/*	Signature verified	Webhook signature check

5. Data Models & Field Specifications

5.1 User

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────
id              UUID        Auto      Generated
name            String      Yes       min:2, max:100
email           String      No        valid email format, unique
mobile          String      Yes       Indian mobile: /^[6-9]\d{9}$/, unique
mobileVerified  Boolean     Auto      default false
passwordHash    String      No        bcrypt 12 rounds (null for OTP-only users)
role            Enum        Auto      ADMIN | EMPLOYEE | USER
subRole         Enum        No        Required if role=EMPLOYEE
isActive        Boolean     Auto      default true
createdAt       DateTime    Auto      IST timezone
updatedAt       DateTime    Auto      IST timezone
```

5.2 Booking

Field	Type	Required	Validation
id	UUID	Auto	Generated
bookingNumber	String	Auto	WP + cuid() — unique, displayed to guest
userId	UUID	No	FK to User (null for walk-in bookings)
guestName	String	Yes	min:2, max:100
guestMobile	String	Yes	/^[6-9]\d{9}$/
guestEmail	String	No	valid email or empty
visitDate	DateTime	Yes	Must be today or future date, within 90 days
adults	Integer	Yes	min:1, max:50
children	Integer	Yes	min:0, max:50
subtotal	Decimal	Auto	Calculated: sum of ticket prices
gstAmount	Decimal	Auto	Calculated: subtotal * gst_rate / 100
discountAmount	Decimal	Auto	Applied coupon or loyalty redemption
totalAmount	Decimal	Auto	subtotal + gstAmount - discountAmount
status	Enum	Auto	PENDING > CONFIRMED > CHECKED_IN > COMPLETED
checkedInAt	DateTime	No	Set on QR scan at gate
createdAt	DateTime	Auto	

5.3 Transaction

Field	Type	Required	Validation
id	UUID	Auto	
bookingId	UUID	Yes	FK to Booking
amount	Decimal	Yes	min:1, must match booking totalAmount
method	Enum	Yes	GATEWAY | MANUAL_UPI | CASH | WRISTBAND | COMPLIMEN
status	Enum	Auto	PENDING > PAID | FAILED | REFUNDED | REJECTED
gatewayRef	String	No	Razorpay order_id
paymentId	String	No	Razorpay payment_id (after success)
upiRef	String	No	Customer-entered UTR/UPI reference (manual UPI)
upiScreenshot	String	No	File path to uploaded screenshot
verifiedBy	UUID	No	FK to User (staff who verified manual UPI)
verifiedAt	DateTime	No	
notes	String	No	max:500
createdAt	DateTime	Auto	

5.4 TicketType

Field	Type	Required	Validation
id	UUID	Auto	
name	String	Yes	e.g., "Adult Day Pass", "Child Day Pass", "Season P
description	String	No	max:500
price	Decimal	Yes	min:0, max:100000
gstRate	Decimal	Yes	default:18, options: 5 | 12 | 18 | 28
isActive	Boolean	Auto	default:true
validDays	Integer	Yes	default:1 (how many park days the ticket covers)

```
minAge          Integer     No        Minimum age for this ticket type
maxAge          Integer     No        Maximum age (e.g., child tickets)
```

5.5 Ride

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────────
id              UUID        Auto
name            String      Yes       min:2, max:100
description     String      No        max:1000
zoneId          UUID        Yes       FK to Zone
minHeight       Integer     No        cm (e.g., 120 for height-restricted rides)
maxWeight       Integer     No        kg
durationMin     Integer     Yes       default:5, min:1, max:120
capacity        Integer     Yes       min:1, max:500 (per session)
status          Enum        Yes       ACTIVE | MAINTENANCE | CLOSED | SEASONAL
imageUrl        String      No        Relative path to uploaded image
currentQueue    Integer     Auto      Redis key: ride:{id}:queue
operatorId      UUID        No        FK to User (assigned operator)
```

5.6 Lead

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────────
id              UUID        Auto
name            String      Yes       min:2, max:100
mobile          String      Yes       /^[6-9]\d{9}$/
email           String      No        valid email
source          Enum        No        WEBSITE | WHATSAPP | PHONE | WALKIN | SOCIAL | REFER
stage           Enum        Auto      NEW > CONTACTED > INTERESTED > PROPOSAL_SENT > BOOKI
groupSize       Integer     No        Expected group size
visitDateExpected DateTime  No        Expected visit date
budgetEstimate  Decimal     No        Estimated booking value
notes           String      No        max:2000
assignedTo      UUID        No        FK to User (sales executive)
followUpAt      DateTime    No        Scheduled follow-up datetime
lostReason      String      No        Required if stage=LOST
createdAt       DateTime    Auto
updatedAt       DateTime    Auto
```

5.7 GuestProfile

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────────
id              UUID        Auto
mobile          String      Yes       unique — primary identifier
name            String      Yes
email           String      No
dob             Date        No        For birthday automation
anniversaryDate Date        No        For anniversary automation
totalVisits     Integer     Auto      Incremented on each completed booking
```

```
totalSpend      Decimal     Auto      Cumulative spend across all bookings
loyaltyPoints   Integer     Auto      Current balance
tier            Enum        Auto      BRONZE | SILVER | GOLD | PLATINUM
lastVisitDate   Date        Auto
tags            String[]    No        Custom tags: VIP, Corporate, Regular
preferredTicket UUID        No        FK to TicketType
createdAt       DateTime    Auto
```

5.8 Wristband

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────
id              UUID        Auto
rfidTag         String      Yes       unique, scanned from RFID reader
bookingId       UUID        Yes       FK to Booking
guestName       String      Yes
balance         Decimal     Auto      default:0, min:0 — cashless wallet
isActive        Boolean     Auto      default:true, deactivated on park exit
activatedAt     DateTime    No        Set on entry scan
deactivatedAt   DateTime    No        Set on exit scan
topupTotal      Decimal     Auto      Cumulative amount added
spendTotal      Decimal     Auto      Cumulative amount spent
```

5.9 FbOutlet

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────
id              UUID        Auto
name            String      Yes       min:2, max:100
location        String      No        e.g., "Wave Pool Area", "Main Entrance"
isActive        Boolean     Auto      default:true
operatingHours  String      No        e.g., "10:00-17:00"
```

5.10 FbMenuItem

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────
id              UUID        Auto
outletId        UUID        Yes       FK to FbOutlet
name            String      Yes       min:2, max:100
category        String      Yes       e.g., "Snacks", "Beverages", "Meals", "Desserts"
price           Decimal     Yes       min:0
gstRate         Decimal     Yes       default:5 (food GST), options: 0 | 5 | 12 | 18
isVeg           Boolean     Yes       default:true
isAvailable     Boolean     Yes       default:true
imageUrl        String      No
description     String      No        max:500
```

5.11 Locker

Field	Type	Required	Validation
id	UUID	Auto	
number	String	Yes	unique, e.g., "A-001", "B-015"
size	Enum	Yes	SMALL | MEDIUM | LARGE
zone	String	Yes	e.g., "Main Entrance", "Wave Pool"
pricePerDay	Decimal	Yes	min:0
isActive	Boolean	Auto	default:true
currentStatus	Enum	Auto	AVAILABLE | OCCUPIED | MAINTENANCE

5.12 Maintenance Asset

Field	Type	Required	Validation
id	UUID	Auto	
name	String	Yes	min:2
assetType	Enum	Yes	RIDE | PUMP | ELECTRICAL | HVAC | STRUCTURAL | VEHIC
location	String	Yes	
serialNumber	String	No	unique if provided
purchaseDate	Date	No	
warrantyExpiry	Date	No	
lastServiceDate	Date	No	
nextServiceDate	Date	No	Auto-calculated or manually set
isActive	Boolean	Auto	default:true

5.13 Work Order

Field	Type	Required	Validation
id	UUID	Auto	
assetId	UUID	Yes	FK to MaintenanceAsset
title	String	Yes	min:5, max:200
description	String	No	max:2000
priority	Enum	Yes	CRITICAL | HIGH | MEDIUM | LOW
status	Enum	Auto	OPEN > IN_PROGRESS > COMPLETED | CANCELLED
assignedTo	UUID	No	FK to User (maintenance tech)
dueDate	DateTime	No	Required if priority=CRITICAL or HIGH
completedAt	DateTime	No	
resolutionNotes	String	No	max:2000
createdBy	UUID	Auto	FK to User

5.14 Campaign

Field	Type	Required	Validation
id	UUID	Auto	
name	String	Yes	min:3, max:100
channel	Enum	Yes	SMS | WHATSAPP | EMAIL | PUSH

```
templateId      UUID        Yes       FK to MessageTemplate
segmentId       UUID        No        FK to GuestSegment (null = all guests)
status          Enum        Auto      DRAFT > SCHEDULED > SENDING > SENT | FAILED | CANCELI
scheduledAt     DateTime    No        Required if not sending immediately
sentAt          DateTime    No
totalRecipients Integer     Auto
sentCount       Integer     Auto
failedCount     Integer     Auto
createdBy       UUID        Auto      FK to User
```

5.15 Coupon

```
Field           Type        Required  Validation
──────────────────────────────────────────────────────────────
id              UUID        Auto
code            String      Yes       uppercase, alphanumeric, unique, min:4, max:20
discountType    Enum        Yes       PERCENTAGE | FLAT
discountValue   Decimal     Yes       if PERCENTAGE: max:100, if FLAT: max:totalAmount
minOrderAmount  Decimal     No        Minimum booking amount to apply coupon
maxDiscountCap  Decimal     No        Max discount amount (for percentage coupons)
maxUses         Integer     No        null = unlimited
usedCount       Integer     Auto      default:0
validFrom       DateTime    Yes       Must be today or future
validTo         DateTime    Yes       Must be after validFrom
isActive        Boolean     Auto      default:true
applicableFor   Enum        No        ALL | TICKETS_ONLY | FB_ONLY | EVENTS_ONLY
```

6. API Specification

6.1 Auth APIs

```
POST /api/v1/auth/register
Body: { name, mobile, email?, password }
Validation: registerSchema (Zod)
Response: { user: {id, name, mobile, role}, token }

POST /api/v1/auth/login
Body: { mobile, password }
Validation: loginSchema (Zod)
Response: { user: {id, name, role}, token }

POST /api/v1/auth/verify-otp
Body: { mobile, otp }
Response: { user, token }
Note: OTP sent via MSG91, stored in Redis with 5-min TTL

POST /api/v1/auth/refresh
Headers: Authorization: Bearer <token>
Response: { token } (refreshed JWT)

POST /api/v1/auth/logout
```

```
Headers: Authorization: Bearer <token>
Action: Invalidate token in Redis blacklist
```

6.2 Booking APIs

```
GET  /api/v1/bookings
Query: page, limit, status, visitDate, search (name/mobile/bookingNumber)
Auth: ADMIN | EMPLOYEE(TICKET_COUNTER)
Response: { bookings: [...], total, page, limit }

POST /api/v1/bookings
Body: { guestName, guestMobile, guestEmail?, visitDate, adults, children, ticketTypeId,
Validation: bookingSchema (Zod)
Auth: PUBLIC (guest booking) or ADMIN/EMPLOYEE
Action: Create booking with PENDING status, calculate totals with GST
Response: { booking, paymentOptions }

GET  /api/v1/bookings/:id
Auth: Owner USER | ADMIN | EMPLOYEE
Response: { booking with transactions }

POST /api/v1/bookings/:id/checkin
Body: { qrCode }
Auth: ADMIN | EMPLOYEE(SECURITY_STAFF | TICKET_COUNTER)
Action: Verify QR, change status to CHECKED_IN, record checkedInAt
Response: { success, booking }

PUT  /api/v1/bookings/:id/cancel
Auth: ADMIN only
Body: { reason }
Action: Change status to CANCELLED, trigger refund if applicable
```

6.3 Payment APIs

```
POST /api/v1/payments/create-order
Body: { bookingId }
Auth: Any authenticated user (booking owner or admin)
Action: Create Razorpay order, store orderId in transaction
Response: { orderId, amount, currency, key }

POST /api/v1/payments/verify
Body: { orderId, paymentId, signature, bookingId }
Action: Verify Razorpay signature, update transaction to PAID,
        update booking to CONFIRMED, send confirmation SMS + email
Response: { success, booking }

POST /api/v1/payments/manual-upi/submit
Body: { bookingId, upiRef, amount, screenshot? }
Validation: manualUpiSchema (Zod)
Auth: Guest (booking owner)
Action: Store UPI reference + screenshot, set transaction PENDING,
        add to UPI verification queue in Redis
Response: { success, message: "Payment submitted for verification" }
```

```
POST /api/v1/payments/manual-upi/verify
Body: { transactionId, action: "APPROVE" | "REJECT", notes? }
Auth: ADMIN only
Action: If APPROVE: set PAID, confirm booking, send confirmation
        If REJECT: set REJECTED, notify guest
Response: { success }

GET  /api/v1/payments/upi-queue
Auth: ADMIN only
Response: { pending: [...transactions with booking details] }
```

6.4 Ride APIs

```
GET  /api/v1/rides
Query: zoneId, status
Auth: PUBLIC
Response: { rides: [...with zone and current queue] }

PUT  /api/v1/rides/:id/status
Body: { status: "ACTIVE" | "MAINTENANCE" | "CLOSED" }
Auth: ADMIN | EMPLOYEE(RIDE_OPERATOR)
Action: Update ride status, broadcast via Redis pub/sub for real-time updates
Response: { ride }
```

6.5 CRM APIs

```
GET  /api/v1/crm/guests
Query: search, tier, tags, page, limit
Auth: ADMIN
Response: { guests: [...], total }

GET  /api/v1/crm/guests/:id
Auth: ADMIN
Response: { guest with bookings, loyalty transactions, communication history }

POST /api/v1/crm/leads
Body: { name, mobile, email?, source, groupSize?, notes? }
Validation: leadSchema (Zod)
Auth: ADMIN | EMPLOYEE(SALES_EXECUTIVE)
Response: { lead }

PUT  /api/v1/crm/leads/:id
Body: { stage?, notes?, followUpAt?, assignedTo? }
Auth: ADMIN | EMPLOYEE(SALES_EXECUTIVE)
Response: { lead }
```

6.6 Analytics APIs

```
GET  /api/v1/analytics
Query: dateFrom, dateTo
Auth: ADMIN
Response: {
  totalBookings, confirmedBookings, cancelledBookings,
  totalRevenue, gatewayRevenue, manualUpiRevenue, cashRevenue,
  totalGuests, newGuests, returningGuests,
  totalLeads, convertedLeads, conversionRate,
  averageSpendPerVisitor,
  peakDayOfWeek, peakHour
}

GET  /api/v1/reports/revenue
Query: period (daily|weekly|monthly), dateFrom, dateTo
Auth: ADMIN
Response: { data: [{date, revenue, bookings, guests}] }

GET  /api/v1/reports/footfall
Query: dateFrom, dateTo
Auth: ADMIN
Response: { data: [{date, adults, children, total}] }
```

6.7 Webhook APIs

```
POST /api/v1/webhooks/razorpay
Headers: X-Razorpay-Signature
Action: Verify signature with RAZORPAY_WEBHOOK_SECRET,
        handle payment.captured, payment.failed, refund.created events

POST /api/v1/webhooks/whatsapp
Action: Handle incoming WhatsApp messages for chatbot responses
```

7. Validation Rules (Zod Schemas)

7.1 loginSchema

```
mobile: z.string().regex(/^[6-9]\d{9}$/, "Invalid Indian mobile number")
password: z.string().min(6, "Password must be at least 6 characters")
```

7.2 registerSchema

```
name: z.string().min(2).max(100)
mobile: z.string().regex(/^[6-9]\d{9}$/)
email: z.string().email().optional().or(z.literal(""))
password: z.string()
  .min(8, "Minimum 8 characters")
```

```
  .regex(/[A-Z]/, "Must contain uppercase")
  .regex(/[0-9]/, "Must contain number")
  .regex(/[!@#$%^&*]/, "Must contain special character")
```

7.3 bookingSchema

```
guestName: z.string().min(2).max(100)
guestMobile: z.string().regex(/^[6-9]\d{9}$/)
guestEmail: z.string().email().optional().or(z.literal(""))
visitDate: z.string().datetime()
  .refine(d => new Date(d) >= new Date(), "Visit date must be today or future")
  .refine(d => {
    const diff = (new Date(d).getTime() - Date.now()) / (1000*60*60*24)
    return diff <= 90
  }, "Cannot book more than 90 days in advance")
adults: z.number().int().min(1).max(50)
children: z.number().int().min(0).max(50)
ticketTypeId: z.string().uuid()
couponCode: z.string().optional()
```

7.4 manualUpiSchema

```
bookingId: z.string().uuid()
upiRef: z.string()
  .min(6, "UTR number must be at least 6 characters")
  .max(50)
  .regex(/^[A-Za-z0-9]+$/, "UTR must be alphanumeric")
amount: z.number().positive()
screenshot: z.string().optional()  // base64 or file path
```

7.5 leadSchema

```
name: z.string().min(2).max(100)
mobile: z.string().regex(/^[6-9]\d{9}$/)
email: z.string().email().optional().or(z.literal(""))
source: z.enum(["WEBSITE","WHATSAPP","PHONE","WALKIN","SOCIAL","REFERRAL","EVENT"]).op
groupSize: z.number().int().min(1).max(10000).optional()
notes: z.string().max(2000).optional()
```

7.6 workOrderSchema

```
assetId: z.string().uuid()
title: z.string().min(5).max(200)
description: z.string().max(2000).optional()
priority: z.enum(["CRITICAL","HIGH","MEDIUM","LOW"])
dueDate: z.string().datetime().optional()
  .superRefine((val, ctx) => {
```

```
    // dueDate required if priority is CRITICAL or HIGH
  })
```

8. Business Logic Rules

8.1 Booking Price Calculation

```
subtotal = adults * adultTicketPrice + children * childTicketPrice
gstAmount = subtotal * (gstRate / 100)
discountAmount = coupon applied OR loyalty points redeemed
totalAmount = subtotal + gstAmount - discountAmount
```

8.2 Loyalty Points System

- Earn: 1 point per Rs.1 spent on tickets and F&B
- Redeem: 1 point = Rs.0.50 discount (max 20% of booking value)
- Tiers:
  - BRONZE: 0 - 999 points (default)
  - SILVER: 1000 - 4999 points (5% discount on next visit)
  - GOLD: 5000 - 14999 points (10% discount + priority entry)
  - PLATINUM: 15000+ points (15% discount + free locker + VIP lane)

8.3 Manual UPI Verification Flow

1. Guest submits UPI reference + optional screenshot
2. Transaction set to PENDING, added to Redis queue
3. Admin sees pending items on UPI Queue dashboard (auto-refreshes every 30s)
4. Admin clicks APPROVE or REJECT
5. On APPROVE: booking CONFIRMED, SMS + email sent to guest
6. On REJECT: guest notified with reason, advised to retry
7. Unverified payments auto-expire after 24 hours (Redis TTL)

8.4 Capacity Management

- Park has a `max_capacity` config value (e.g., 2000 guests/day)
- Each CONFIRMED booking increments Redis counter: `park:capacity:{date}`
- Booking fails if counter >= max_capacity
- Admin can override capacity limit from settings
- Counter resets at midnight via scheduled job

8.5 Ride Queue Management

- Each ride has real-time queue count in Redis: `ride:{id}:queue`
- Staff RIDE_OPERATOR increments/decrements queue via staff panel
- Guest app shows live wait time (queue * avgDurationMin)
- Queue data broadcast via Server-Sent Events (SSE) to admin dashboard

8.6 RFID Wristband Cashless Flow

1. Guest activates wristband at entry gate (linked to booking)
2. Guest tops up balance at kiosk (GATEWAY or CASH)
3. At F&B / activity point, RFID scanned → balance deducted
4. Minimum balance alert at Rs.50
5. On park exit, remaining balance refunded to Razorpay/UPI
6. Wristband deactivated on exit

9. Data Storage Architecture

9.1 PostgreSQL — What Stored

Schema Area	Tables	Storage Estimate (1 year, 50k bookings)
Users & Auth	users, sessions	~50 MB
Bookings & Payments	bookings, transactions	~500 MB
Guest CRM	guest_profiles, loyalty_transactions	~200 MB
Rides & Zones	rides, zones, ride_access_logs	~100 MB
F&B	fb_outlets, fb_menu_items, fb_orders	~300 MB
Lockers	lockers, locker_rentals	~50 MB
Maintenance	assets, work_orders, inspections	~100 MB
CRM & Leads	leads, lead_activities	~150 MB
Campaigns	campaigns, templates, comm_logs	~500 MB
Staff	shifts, attendance	~100 MB
Total	30+ tables	~2 GB

9.2 Redis — What Stored

Key Pattern	Data	TTL
`session:{userId}`	JWT token blacklist	24 hours
`otp:{mobile}`	6-digit OTP	5 minutes
`park:capacity:{date}`	Current booking count	Until midnight
`ride:{id}:queue`	Current queue count	No expiry
`upi:queue`	List of pending UPI transaction IDs	24 hours
`cache:analytics:{date}`	Analytics response cache	1 hour
`cache:rides:list`	Active rides list	5 minutes
`cache:tickets:list`	Ticket types list	10 minutes
`ratelimit:{ip}`	API rate limit counter	1 minute

9.3 File Storage — Uploads

All uploaded files stored in Docker volume `/app/uploads`:

```
/app/uploads/
├── screenshots/       UPI payment screenshots (jpg, png — max 5MB)
├── rides/             Ride images (jpg, png — max 2MB)
├── food/              Menu item images (jpg, png — max 1MB)
├── staff/             Staff profile photos (jpg, png — max 1MB)
└── events/            Event package images (jpg, png — max 2MB)
```

File naming convention: `{type}-{uuid}.{ext}` (e.g., `screenshot-abc123.jpg`)

Files served directly by Nginx with `expires 30d` header for images.

10. Security Implementation

10.1 Authentication Security

Measure	Implementation
Password hashing	bcrypt with 12 salt rounds
JWT secret	48-byte random string via `openssl rand -base64 48`
JWT expiry	24 hours (access token)
OTP expiry	5 minutes, max 3 attempts before lockout
Failed login throttle	5 attempts → 30-minute lockout (Redis-based)
Session invalidation	Token blacklist in Redis on logout

10.2 API Security

Measure	Implementation
Input validation	Zod on all POST/PUT routes (body + query params)
SQL injection	Prisma ORM parameterized queries (no raw SQL)
Rate limiting	100 req/min per IP for public routes, 1000 for authenticated
CORS	Restricted to NEXTAUTH_URL origin only
Webhook verification	HMAC-SHA256 signature check for Razorpay webhooks
File upload	Type check (image/* only), size limit (5MB), stored outside webroot
Encryption	AES-256-GCM for sensitive fields (UPI references)

10.3 Infrastructure Security

Measure	Implementation
Firewall	UFW: only 22, 80, 443 open to internet
SSH hardening	Root login disabled, password auth disabled, key-only
Database binding	PostgreSQL on 127.0.0.1 only, never exposed
Redis binding	Redis on 127.0.0.1 with password
SSL	TLS 1.2 + 1.3 only, strong cipher suite
Security headers	HSTS, X-Frame-Options, X-Content-Type-Options, CSP
Non-root containers	Next.js runs as `nextjs` user (uid 1001)
Brute force protection	Fail2ban on SSH

11. Environment Variables Reference

```
# ── DATABASE ──────────────────────────────────────
POSTGRES_DB              Database name (waterpark_db)
POSTGRES_USER            Database user (waterpark_user)
POSTGRES_PASSWORD        Strong random password (min 32 chars)
DATABASE_URL             Full connection string for Prisma

# ── REDIS ─────────────────────────────────────────
REDIS_PASSWORD           Strong random password
REDIS_URL                redis://:password@redis:6379

# ── APP ───────────────────────────────────────────
NODE_ENV                 production
NEXTAUTH_URL             https://yourdomain.com
NEXTAUTH_SECRET          48-byte random string (openssl rand -base64 48)

# ── PAYMENT ───────────────────────────────────────
RAZORPAY_KEY_ID          From Razorpay dashboard
```

```
RAZORPAY_KEY_SECRET      From Razorpay dashboard
RAZORPAY_WEBHOOK_SECRET  Set in Razorpay dashboard webhook settings
NEXT_PUBLIC_RAZORPAY_KEY_ID  Same as RAZORPAY_KEY_ID (client-side)

# ── COMMUNICATIONS ──────────────────────────────────
MSG91_API_KEY            From MSG91 dashboard
MSG91_SENDER_ID          6-char alphanumeric (e.g., WTRPRK)
MSG91_OTP_TEMPLATE_ID    OTP template approved by DLT
MSG91_FLOW_ID            Promotional/transactional flow ID
MSG91_WHATSAPP_NUMBER    WhatsApp Business number

GUPSHUP_API_KEY          From Gupshup dashboard
GUPSHUP_APP_NAME         Registered app name on Gupshup
GUPSHUP_SOURCE_NUMBER    WhatsApp source number

SENDGRID_API_KEY         From SendGrid dashboard
SENDGRID_FROM_EMAIL      Verified sender email
SENDGRID_FROM_NAME       Display name (e.g., WaterWorld Park)

# ── FIREBASE ────────────────────────────────────────
FIREBASE_SERVICE_ACCOUNT JSON string of Firebase service account key

# ── SECURITY ────────────────────────────────────────
ENCRYPTION_KEY           Exactly 32 characters — AES-256 key

# ── UPLOADS ─────────────────────────────────────────
UPLOAD_DIR               /app/uploads (inside container)
MAX_FILE_SIZE_MB         10

# ── PUBLIC (exposed to browser) ─────────────────────
NEXT_PUBLIC_PARK_NAME    Display name of the park
NEXT_PUBLIC_RAZORPAY_KEY_ID  Razorpay public key
```

12. Development Workflow

12.1 Git Branching Strategy

```
main         → Production branch (protected, auto-deploys via GitHub Actions)
develop      → Integration branch (all features merge here first)
feature/*    → Individual feature branches (e.g., feature/manual-upi-flow)
fix/*        → Bug fix branches (e.g., fix/booking-capacity-bug)
hotfix/*     → Emergency production fixes
```

12.2 Commit Message Convention

```
feat: add manual UPI payment verification queue
fix: correct GST calculation for child tickets
refactor: extract payment logic to lib/razorpay.ts
docs: update API specification for booking endpoint
```

```
test: add unit tests for loyalty points calculation
chore: upgrade prisma to 5.12.0
```

12.3 Development Phase Plan

Phase	Duration	Deliverables
Phase 1 — Core Foundation	Weeks 1-4	Auth, Bookings, Online Payment (Razorpay), Admin Dashboard, Check-in
Phase 2 — Operations	Weeks 5-8	F&B POS, Lockers, Ride Management, Staff Module, Manual UPI
Phase 3 — CRM & Marketing	Weeks 9-12	Guest CRM, Lead Pipeline, Campaigns (SMS/WhatsApp/Email), Loyalty
Phase 4 — Advanced	Weeks 13-16	RFID Wristband, Event Packages, Coupons, Reports & Analytics
Phase 5 — Polish	Weeks 17-20	Mobile PWA, Real-time dashboard, Push notifications, Performance

12.4 Testing Requirements

Test Type	Tool	Coverage Target
Unit tests	Jest	Business logic (pricing, loyalty)
Integration tests	Jest + Supertest	All API routes
E2E tests	Playwright	Guest booking flow, Admin workflows
Type checking	TypeScript strict mode	100%
Linting	ESLint + Next.js config	All files

13. Deployment Reference

13.1 Server Minimum Requirements

Resource	Minimum	Recommended
CPU	2 vCPU (ARM64)	4 vCPU
RAM	4 GB	8 GB
Storage	50 GB SSD	100 GB SSD
Bandwidth	1 TB/month	5 TB/month
OS	Ubuntu 24.04 LTS ARM64	Same

13.2 Production Checklist

Before going live:

- [ ] All .env values filled with production credentials (not placeholders)
- [ ] NEXTAUTH_SECRET generated with openssl rand -base64 48
- [ ] ENCRYPTION_KEY set to exactly 32 characters
- [ ] SSL certificate issued and auto-renewal tested
- [ ] Database backup cron job running (crontab -l to verify)
- [ ] Firewall enabled with only 22/80/443 open
- [ ] Fail2ban active
- [ ] Default admin password changed from Admin@1234
- [ ] Razorpay webhook URL configured in Razorpay dashboard
- [ ] MSG91 DLT templates approved
- [ ] Test booking + payment flow end-to-end
- [ ] Test manual UPI submit + admin verification flow
- [ ] All Docker containers showing healthy status

13.3 Monitoring Commands

```
# Container health
docker compose ps
docker stats

# App logs (live)
docker compose logs -f app

# PostgreSQL connections
docker compose exec postgres psql -U waterpark_user -d waterpark_db   -c "SELECT count(*

# Redis memory usage
docker compose exec redis redis-cli -a $REDIS_PASSWORD info memory

# Disk usage
df -h
du -sh /opt/waterpark-app/

# SSL expiry
sudo certbot certificates
```

14. Admin Dashboard — Required KPIs

14.1 Top-Level Stats (visible on dashboard home)

KPI	Calculation	Refresh
Today's Bookings	COUNT bookings WHERE visitDate=today	1 min
Today's Revenue	SUM transactions WHERE status=PAID AND date=today	1 min
Current Guests In Park	Redis: park:capacity:{today}	30 sec
Pending UPI Verifications	Redis: LLEN upi:queue	30 sec
Active Rides	COUNT rides WHERE status=ACTIVE	5 min
Open Work Orders	COUNT work_orders WHERE status=OPEN	5 min

14.2 Reports Required

Report	Dimensions	Export
Revenue Report	Daily/Weekly/Monthly, by payment method	CSV, PDF
Footfall Report	Daily visitors, adults vs children ratio	CSV
Ride Popularity	Ride-wise visitor count	CSV
F&B Sales Report	Outlet-wise, item-wise revenue	CSV
Lead Conversion	Source-wise, stage-wise pipeline	CSV
Loyalty Summary	Points issued, redeemed, balance	CSV
Staff Attendance	Date-wise, shift-wise	CSV
Maintenance Log	Asset-wise work order history	CSV

15. Communication Templates

15.1 SMS Templates (DLT registered)

Booking Confirmation:

```
Dear {name}, your booking {bookingNumber} for {parkName} on {visitDate}
is confirmed. Adults: {adults}, Children: {children}. Total: Rs.{amount}.
Show QR at entry. - {parkName}
```

Payment Reminder (Manual UPI):

```
Your booking {bookingNumber} is awaiting payment verification.
Please ensure UTR {upiRef} matches your UPI transaction.
Queries: {supportNumber} - {parkName}
```

OTP:

```
Your OTP for {parkName} login is {otp}. Valid for 5 minutes.
Do not share with anyone. - {parkName}
```

Birthday Offer:

```
Happy Birthday {name}! Celebrate at {parkName} with 20% OFF.
Use code BDAY{year} valid till {expiryDate}. Book: {websiteUrl} - {parkName}
```

15.2 Email Templates

- Booking confirmation (HTML with QR code embedded)
- Payment success receipt (HTML with itemized breakdown)
- Manual UPI approval/rejection notification
- Password reset link
- Monthly loyalty points statement

15.3 WhatsApp Templates (Gupshup approved)

- Booking confirmation with QR code image
- Visit reminder (1 day before)
- Feedback request (1 day after visit)
- Lead follow-up sequences (Day 1, Day 3, Day 7)